QA TEST AUTOMATION

Web & Mobile App Test Automation Portfolio

e커머스 플랫폼 웹 + Android/iOS 앱 크로스 플랫폼 E2E 자동화 테스트

Playwright | Appium | Python | Pytest | Allure | Slack

39 Scenarios | 256 Test Steps | Web + Android + iOS | Real-time Slack Alert | Allure HTML Report

테스트 자동화 전략 수립 및 방법론

자동화 대상 선정 기준

적합한 대상

- 주기적/반복적 실행이 필요한 테스트
- 수동 진행이 복잡하거나 시간 소모가 큰 테스트
- 동시성·성능·메모리 누수 탐지 테스트
- 다양한 데이터 조합이 필요한 테스트

적합하지 않은 대상

- 요구사항이 자주 변경되는 테스트
- 한 번만 실행되는 일회성 테스트
- 아직 수동 검증이 안 된 신규 테스트

자동화 프로세스

툴 선택 → 범주 정의 → 설계·개발 → 테스트 실행 → 결과 판단 → 유지보수

활용 경험 도구

Selenium · Appium · Playwright · TestComplete · Katalon · JMeter · AutoIt · Sikuli · Robot Framework

테스트 자동화 접근 방법 비교

GUI 기반 (외부 동작)

마우스/키보드 조작, 이미지 매칭, 좌표 기반 입력

장점 범용성 높음, 프로그램 이해만으로 구현 가능
단점 PC 환경 영향, 화면 포커스 점유, 정확도 편차
도구 AutoIt · Sikuli · Guitar · 매크로 프로그램

API 기반 (내부 동작)

HTTP/REST API, 내부 함수 호출 기반 자동화

장점 별도 실행 가능, 높은 정확도, 디버깅 용이
단점 개발자 지원 필수, 사용 용도 한정적
도구 Selenium · Appium · Playwright · JMeter

핵심 목표

저비용으로 효율 극대화, 수동 테스트와 병행하여 커버리지 확대

프로젝트 개요

Web - Playwright 기반

e커머스 플랫폼 웹사이트의 핵심 사용자 여정을 Playwright 기반으로 자동화하여 회귀 테스트를 수행합니다.

URL https://ecommerce-platform.com
Browser Chromium (headless/headed)

12개 시나리오 / 100+ 테스트 스텝 / ~8분 실행

모바일 앱 - Appium 기반

e커머스 플랫폼 모바일 앱의 핵심 사용자 여정을 Appium + UiAutomator2 기반으로 자동화하여 회귀 테스트를 수행합니다.

App e커머스 플랫폼
Engine UiAutomator2 (Appium 2.x)

14개 시나리오 / 80+ 테스트 스텝 / ~25분 실행

39	256	~25분	Slack	Allure
총 시나리오	총 테스트 스텝	전체 실행 시간	실시간 알림	HTML 리포트

기술 스택

공통 코어

1 Python 3.x

테스트 스크립트 작성 언어
높은 가독성, 풍부한 라이브러리

2 Pytest

테스트 러너 및 fixture 관리
allure-pytest 플러그인 통합

3 Allure Report

인터랙티브 HTML 리포트
스텝별 상세 결과 + 스크린샷

4 Slack API

실시간 알림 + 스레드 보고
성공/실패 결과 자동 전송

5 Fernet 암호화

계정 정보 AES-128 암호화
.env + CSV 이중 보안

Web 전용

P Playwright

- 최신 브라우저 자동화 프레임워크
- Chromium/Firefox/WebKit 멀티 브라우저 지원
- auto-wait, 네트워크 인터셉트, React SPA 완벽 대응

모바일 앱 전용

A Appium 2.x

- 모바일 앱 테스트 자동화 프레임워크
- Android(UiAutomator2)
- iOS(XCUITest)

전 플랫폼 공통

T Tkinter GUI

- Python 기반의 GUI 라이브러리
- Web/Android/iOS 전 플랫폼 테스트 선택 실행

Python 실전 자동화 코드 역량

Selenium 드라이버 관리

Chrome 드라이버 자동 다운로드
버전 업데이트 호환성 문제 자동 해결
설치 경로 커스터마이징 지원

웹 로그인 봇 감지 회피

네이버: pyperclip + pyautogui 로 Captcha 우회
구글: undetected_chromedriver로 봇 감지 회피

모바일 자동화 (Appium)

UIAutomator2 키 이벤트로 검색 실행
동영상 녹화 (Android 3분 / iOS 10분)
텍스트 기반 스크롤 탐색 함수 구현
Android 버전별 스크롤 자동 보정

데이터 처리

Pandas CSV 읽기/저장 자동화
UTF-8 한글 인코딩 처리 (utf-8-sig)
정규식으로 특수문자·이모지 제거
파일명 타임스탬프 자동 삽입

배포 및 패키징

PyInstaller로 단일 파일 배포
Windows exe / Mac app 크로스 플랫폼
라이브러리 종속성 자동 번들링

Slack API 연동

conversations.history로 메시지 수집
conversations.replies로 스레드 파싱
chat.postMessage로 알림 전송
파일 업로드 (PDF, 리포트)

프로젝트 구조 - Web

디렉토리 구조

```
pom_web_re/
  ci_web_test_run.py        Pytest 테스트 러너
  create_report.py          Allure 리포트 생성
  .env                      암호화된 환경변수

  ci_test_scenario/
    web_login.py            로그인 테스트
    web_signup.py           회원가입 테스트
    web_main_theme.py       메인 테마 테스트
    web_inbox.py            인박스 테스트
    web_gig_order.py        구매/결제 테스트
    ...외 7개 시나리오

  common/
    web_common_func.py      공통 유틸리티
    web_run_playwright.py   브라우저 관리 (싱글톤)
    security_text.py        Fernet 암/복호화
    slack_test.py           Slack 메시지 전송
    test_account.csv        암호화된 계정 정보
```

실행 아키텍처

Pytest Runner

ci_web_test_run.py | fixture: setup / teardown

↓

Test Scenarios

12개 시나리오 모듈 | 각 시나리오 독립 실행

↓

Common Layer

web_common_func | web_run_playwright

↓

Playwright

Chromium 브라우저 제어 | auto-wait

↓

Report & Alert

Allure HTML | Slack 실시간 알림 | 스크린샷

프로젝트 구조 - Android

디렉토리 구조

```
ci_pom_android_re/
  android_test_run.py          Pytest 테스트 러너
  test_run_ui.py               Tkinter GUI 실행기
  create_report.py             Allure 리포트 생성
  .env.example                 환경변수 설정

  test_base/
    app_context.py             AppContext (드라이버+요소+설정)
    android_driver_base.py     드라이버 팩토리
    android_driver_real.py     실기기 드라이버
    android_driver_jenkins.py  Jenkins Device Farm

  test_scenario/
    android_login.py           로그인 테스트
    android_signup.py          회원가입 테스트
    android_order.py           구매/결제 테스트
    ...외 14개 시나리오

  common/
    android_common_func.py     공통 유틸리티 (625줄)
    android_element.py         POM 요소 관리 (560줄)
    android_element.csv        CSV 로케이터 (186행)
    security_text.py           Fernet 암/복호화
    slack_test.py              Slack 메시지/파일 전송
```

실행 아키텍처



프로젝트 구조 - iOS

디렉토리 구조

```
pom_ios/
  test_main.py              Pytest 테스트 러너 (14 tests)
  create_report.py          Allure 리포트 생성
  create_pdf_file.py        PDF 보고서 변환
  ios_pipeline.jenkinsfile  Jenkins CI/CD 파이프라인

  app_base/
    base.py                 AppBase (658줄, 핵심 드라이버 조작)
    context.py              AppContext (의존성 주입 컨테이너)
    element.py              UI 요소 로케이터 (266개 요소)
    sift_image_compare.py   SIFT/RANSAC 이미지 매칭
    ios_slack_common.py     Slack 스레드 기반 리포팅
    ios_device_info.py      iPhone 11~17 해상도 DB

  test_base/
    driver_manager.py       get_driver(env) 팩토리
    setup_real.py           Device Farm (XCUITest)
    setup_single.py         로컬 Appium 연결

  test_case/                14개 시나리오 모듈
```

실행 아키텍처

Pytest Runner

test_main.py | 14 ordered tests + flaky retry

AppContext

driver + Element + AppBase 통합 객체

Test Scenarios

14개 시나리오 모듈 | run_test_step 체이닝

Appium + XCUITest

AppBase (658줄) | element.py (POM) | Device Farm

Report & Alert

Allure HTML | Slack 스레드 알림 | 스크린샷

테스트 시나리오 - Web (12개)

01 메인 페이지 비로그인

배너, 테마 카테고리 추천 상품, 외부 링크

02 카테고리 비로그인

카테고리별 상품 개수 및 페이지 오류 체크

03 검색 기능 비로그인

상품/판매자/포트폴리오 검색 및 필터 동작

04 회원가입 비로그인

이메일/비밀번호 유효성에 대한 12가지 규칙 검증 후 회원가입

05 회원탈퇴 로그인

가입된 계정으로 회원 탈퇴 프로세스 검증

06 로그인 비로그인

이메일 로그인 성공 여부 확인

07 메인 테마 로그인

개인화 추천 8개 섹션 상품 노출 및 클릭

08 상품 상세 로그인

찜한 상품 상세 리뷰, 가격, 판매자

09 메시지 로그인

메시지 전송, 금칙어 경고, 메모

10 구매/결제 로그인

상품 구매 후 결제 프로세스

11 구매 관리 로그인

주문 상세 내용 확인

12 구매 취소 로그인

구매자 취소 요청 후 판매자 수락 플로우

테스트 시나리오 - 모바일 앱 (14개)

01 초기 설정 비로그인

권한 허용, 인트로 로그인 페이지 진입

02 카테고리 비로그인

1차 카테고리 상품 목록 확인

03 로그인 전 비로그인

상품 테마, 검색, 로그인 리다이렉트

04 회원가입 비로그인

이메일/비밀번호 검증
가입 + 2FA 해제

05 비밀번호 변경 로그인

기존 PW 확인, 새 PW 규칙 검증

06 회원탈퇴 로그인

탈퇴 사유 선택, 2단계 확인 후 탈퇴

07 로그인 비로그인

이메일 로그인, Braze 팝업 처리

08 메인 테마 로그인

9개 상품 테마 섹션, 상품 노출 확인

09 검색 로그인

키워드/필터 검색, 상품/판매자/포폴

10 개인화 위젯 로그인

찜 상품 상세, 4개 추천 위젯

11 인박스 로그인

메시지 전송, 메모, 판매자 프로필 확인

12 구매/결제 로그인

상품 구매 후 결제 프로세스

13 구매 관리 로그인

주문 필터/목록 등 구매 상세 정보 확인

14 노하우 로그인

노하우 페이지 8개 상품 테마

공통 설계 - run_test_step 실행 구조

크로스 플랫폼 공통 실행 아키텍처

`run_test_step(page|context, func, name, results, *args)`
스텝 함수 실행 → 성공 시 PASS / 실패 시 자동 스크린샷 + FAIL
Allure @step 기록 | Slack 실시간 알림 | 결과 배열에 저장

▼

`set_test_scenario_result(results)`
스텝 결과 배열 순회 → 하나라도 실패 시 FAIL / 전부 성공 시 PASS
시나리오 단위 최종 판정

▼

`save_test_result(result, scenario, test_result)`
시나리오 최종 결과를 전체 결과 배열에 저장
Slack 채널 전송 | 기본기능체크.txt 기록

Web vs Android 코드 패턴

```
# Web: web_main_page.py
run_test_step(page, check_banner,
    "1. 배너 확인", results, page)
run_test_step(page, click_theme,
    "2. 테마 카테고리 확인", results, page)

# Android: android_signup.py
run_test_step(context, move_to_signup,
    "1. 회원가입 페이지 이동", results)
run_test_step(context, check_email_value,
    "2. 이메일 입력값 검증", results, email)

# 동일 패턴: 함수 + 이름 + 결과배열
result = set_test_scenario_result(results)
save_test_result(result, scenario, test_result)
```

설계 강점

- **관심사 분리** — 테스트 로직과 실행·기록·알림 책임을 완전 분리
- **확장 용이** — 함수 1개 작성 + run_test_step 1줄 = 새 스텝 완성
- **균일한 품질** — 모든 스텝에 에러처리·스크린샷·Allure·Slack 자동 적용
- **템플릿 패턴** — 시나리오 템플릿 복사 → 즉시 새 시나리오 개발 가능

Web 강점 - Playwright 기반 자동화

Selenium 대비 Playwright 이점

Auto-Wait 내장
요소가 준비될 때까지 자동 대기
time.sleep() 의존도 최소화로 테스트 안정성 향상

다중 탭/팝업 제어
expect_popup()으로 새 탭 자동 캐치
탭 간 전환이 자연스럽고 직관적

React SPA 완벽 대응
nativeInputValueSetter + 이벤트 디스패치
React controlled input에 2단계 폴백 전략

브라우저 컨텍스트 격리
테스트 간 세션/쿠키 완전 분리
테스트 독립성 보장

네트워크 인터셉트
요청/응답 가로채기 내장
외부 의존성 제어 가능

React Input 대응 코드 (web_inbox.py)

```
# 1차: press_sequentially로 React onChange
input_box.press_sequentially(msg, delay=30)

# 전송 버튼 활성화 확인
page.wait_for_function("""
  () => {
    const btns = document.querySelectorAll('button');
    return [...btns].some(
      b => b.textContent.includes('전송')
           && !b.disabled
    );
  }
""", timeout=3000)

# 2차: nativeInputValueSetter 폴백
if not btn_enabled:
    page.evaluate("""
      (msg) => {
        const ta = document.querySelector('textarea');
        const setter = Object.getOwnPropertyDescriptor(
          HTMLTextAreaElement.prototype, 'value'
        ).set;
        setter.call(ta, msg);
        ta.dispatchEvent(
          new Event('input', {bubbles: true})
        );
      }
    """, message)
```

Android 강점 - AppContext & POM & 멀티디바이스

AppContext - 통합 컨텍스트 객체

모든 테스트에 필요한 리소스를 하나의 객체로 통합 관리

COMPONENTS

self.driver	Appium WebDriver
self.app_element	AosElement (POM)
self.test_id/pw	복호화된 테스트 계정
self.device_w/h	디바이스 해상도
self.slack_*	Slack 알림 설정
self.test_result	결과 누적 배열

→ 파라미터 폭발 방지, 의존성 주입 패턴

AosElement - CSV 기반 POM

요소 로케이터를 CSV 파일로 외부 관리 (200+ 행)

화면명 | 항목명 | id | xpath | 구분

set_element / click / send_keys / scroll_to_element

코드 수정 없이 CSV만 변경 → 로케이터 업데이트

3환경 드라이버 팩토리

get_driver(env) 한 줄로 실행 환경 전환

env="real"	실기기 USB (port 4723)
env="single"	에뮬레이터 (port 4725)
env="jenkins"	Jenkins Device Farm (AWS)

Android 9/10/15 버전별 스크롤 자동 보정

iOS 강점 - SIFT 이미지 매칭 & XCUITest

SIFT/RANSAC 이미지 매칭 (iOS 고유)

복잡한 UI 요소를 이미지 기반으로 탐색·탭

SIFT 특징점 검출
OpenCV SIFT 알고리즘으로 템플릿 이미지의 특징점 추출

RANSAC 좌표 매핑
호모그래피 추정으로 정확한 화면 좌표 계산

전처리 파이프라인
명암 대비 향상 + 이미지 업스케일링으로 인식률 개선

폴백 전략
SIFT 실패 시 template matching으로 자동 전환

Braze 인앱 마케팅 자동 처리

braze_popup()
X 버튼 감지 → 닫기 + 뒤로가기 네비게이션

braze_slide()
슬라이드 팝업 감지 → chevron 클릭으로 닫기

braze_check_click()
요소 클릭 후 Braze 팝업 자동 감지 + 처리

AppContext + XCUITest 아키텍처

AppContext = driver + Element + AppBase

driver	Appium WebDriver (XCUITest)
Element	100+ UI 로케이터 (static)
AppBase	658줄 핵심 조작 메서드

→ 파라미터 폭발 방지, 의존성 주입 패턴

멀티 환경 드라이버 팩토리

get_driver(env) 한 줄로 실행 환경 전환

env="real"	Device Farm (RemoteWebDriver)
env="single"	로컬 Appium 서버 (port 4723)

iPhone 11~17 시리즈 해상도 자동 감지 및 좌표 보정

iOS 고유 기능

동영상 녹화	h264 코덱, 30fps, 1280x720, 최대 30분
로케이터	IOS_CLASS_CHAIN · ACCESSIBILITY_ID · XPATH · PREDICATE
키보드 관리	tap_center_screen()으로 자동 키보드 닫기

공통 강점 - 보안 & 안정성

Fernet 암호화 기반 계정 보안

PROBLEM

테스트 계정(ID/PW)이 소스코드에 평문으로 노출

SOLUTION (Web + Android 동일 적용)

1. .env 파일 계정 정보 Fernet(AES-128-CBC) 암호화
2. test_account.csv 전체 값 암호화 저장
3. 런타임에 FERNET_KEY로 복호화하여 사용

STRUCTURE

.env

```
TEST_ID1=gAAAAABppo...
FERNET_KEY=<redacted>      # decrypt key
```

test_account.csv

```
gAAAAABppo...,gAAAAABppo...,gAAAAABppo...
```

ENV 우선, 미설정 시 CSV 폴백 (이중 보안)

안정성 확보 전략 (크로스 플랫폼)

다중 셀렉터 폴백

[Web+AOS] Web: CSS → 대안 셀렉터 자동 전환
Android: id → xpath 자동 폴백

팝업/모달 자동 처리

[Web+AOS] Web: Escape > close-btn > JS 3단계
Android: Braze 4종 팝업 자동 감지+닫기

실패 스크린샷 자동 저장

[Web+AOS] Web: 원본 사이즈 캡처
Android: 400x800 리사이즈 + Allure 첨부

스크롤 탐색

[AOS] Android: scroll_to_element 발견 시까지 재시도
방향 자동 전환 (하→상 폴백)

Graceful Skip

[Web+AOS] 환경 의존 시나리오는 FAIL 대신 SKIP
모든 플랫폼 동일 적용

공통 강점 - 리포팅 시스템

Allure Report 통합

양 플랫폼 동일 파이프라인으로 리포트 자동 생성

PIPELINE

```
1  pytest --alluredir=./allurereports
2  allure generate --clean
3  allure-combine (단일 HTML 병합)
```

FEATURES

시나리오별 PASS/FAIL 대시보드
스텝별 상세 실행 로그 + 실패 시점 스크린샷
실행 시간 통계 및 트렌드

GUI 테스트 러너 (Android)

Tkinter 기반 체크박스 선택 실행 + Braze 팝업 토글
비개발자도 테스트 선택/실행 가능, 리포트 자동 생성

Slack 실시간 알림

스레드 기반 그룹핑 + 시나리오별 성공/실패 실시간 전송
컬러코드 메시지: 성공(green) / 불안정(yellow) / 실패(red)

공통 강점 - CI/CD 파이프라인

Jenkins Pipeline

Multi-Stage 파이프라인

1. Notify Start — Slack 시작 알림 + Jenkins 링크
2. Clone Repo — Git 저장소 최신 코드 체크아웃
3. Execute Shell — pytest + 환경 세팅 실행
4. Post Actions — 리포트 생성 + 결과 판정

3-Retry + 60초 Backoff

일시적 환경 오류 시 자동 재시도 (최대 3회)

오류 유형 자동 분류

테스트 실패 vs 인프라 장애 구분 → 빌드 상태 분리 판정

디바이스 자동 준비 (device_unlock.sh)

Jenkins 실행 전 디바이스 상태 자동 확인/해제

ADB dumpsys → mDreamingLockscreen 감지

keyevent 26 (전원) + swipe 300px (잠금해제)

테스트 시작 전 디바이스 Ready 상태 보장

리포트 자동화 (ci_create_report.py)

allure generate → combine → 날짜별 파일명 자동 부여

android_automation_testreport_YYYYMMDD.html

Allure Report HTML → PDF 자동 변환 시스템

문제 상황

HTML 파일 → 슬랙 보안정책으로 직접 열람 불가
allure-pdf, wkhtmltopdf → 한글 깨짐 발생
JavaScript SPA 기반 → 일반 변환 도구 처리 불가

해결 과정 (Gemini AI 활용)

1	**allure-pdf 시도**	한글 깨짐 → 실패
2	**Puppeteer 시도**	JS 숙련도 부족 → 전환
3	**Playwright Python 선정**	브라우저 렌더링 → 성공
4	**첫 페이지만 변환됨**	탭 순회 클릭 로직 추가
5	**여백·레이아웃 문제**	A3 가로 + scale 0.9 최적화
6	**시나리오 상세 미표시**	node_order 개별 PDF 생성
7	**테스트 스텝 미펼침**	div.step chevron 클릭 구현

핵심 기술 상세

렌더링 엔진	Playwright Python async API + Chromium headless
로컬 파일 접근	file:// URI + --enable-local-file-accesses
탭 네비게이션	CSS 선택자로 Overview / Graphs / Timeline / Behaviors 순회
상세 펼치기	node_order 역순 클릭 → div.step chevron 클릭
PDF 생성	탭별 page.pdf() 생성 → PyPDF2 PdfMerger 병합
순서 보정	역순 클릭 + reversed() append → 페이지 순서 보정
레이아웃	A3 가로, scale=0.9, margin 최적화 적용
자동 파일명	Android_Automation_Test_Report_yyyymmdd.pdf

운영 현황

CI 장비 연동 → 일일 테스트 결과 PDF 자동 전송 운영 중
2주 주기의 모바일 앱 정기배포 자동화 테스트에도 활용
슬랙에서 PDF 즉시 열람 → 리뷰 시간 대폭 단축

일일 테스트 자동화 수행 통계 시스템

시스템 개요

일일 테스트 자동화 결과를 월별로 자동 집계/분석
Slack API로 채널 메시지 + 스레드 자동 수집
실패 원인 분석 내용 자동 추출 및 리포팅
Excel 파일로 월별 통계 자동 저장 (openpyxl)

핵심 통계 지표

일일 성공률

성공일 수 / 수행일 수

전체 수행일 중 전 시나리오 PASS 달성 비율

시나리오 성공률

Passed / (Total - Skipped)

Skipped 제외한 시나리오 중 Passed 비율 (월간 집계)

기술 구현

Slack API + 커서 기반 페이지네이션
conversations.history + replies (200건 초과 자동 처리)

정규식 파싱
Passed / Failed / Broken / Skipped 자동 추출

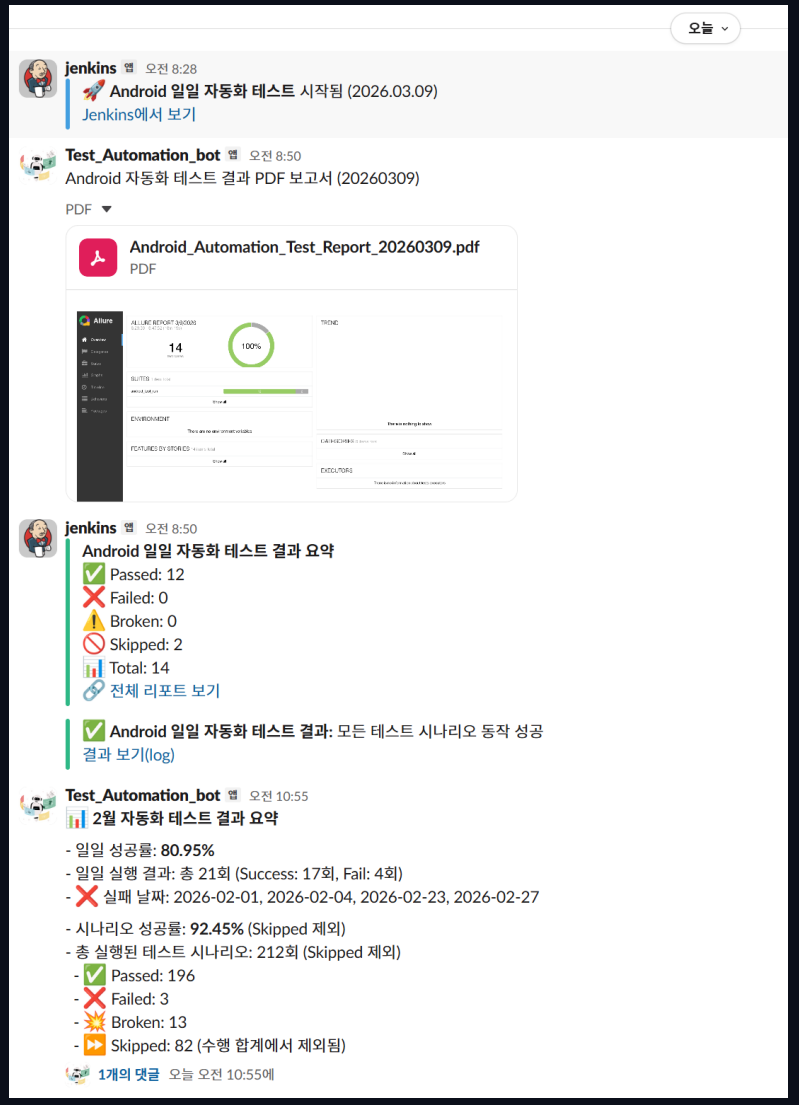
Pandas + openpyxl
Excel 월별 시트 자동 저장 (append + replace)

전월 대비 분석
월간 성공률 증감(Δ) 계산 + Slack 리포트 자동 발송

실패 원인 분석 자동 수집

Slack 스레드에서 실패 원인 코멘트 자동 수집
정규식으로 에러 메시지 / 원인 분류 추출
월별 실패 유형 통계 자동 집계
Excel 시트에 원인별 빈도 기록

자동화 테스트 실행 증적 - Slack 리포팅



자동화 파이프라인 연계

Jenkins Cron → Pytest 실행
매일 오전 자동 트리거

Pytest → Allure → PDF
테스트 결과 리포트 자동 변환

PDF + 요약 → Slack API
채널 업로드 + 스레드 기반 알림

연동 시스템

Jenkins	CI 스케줄러 + 빌드 관리
Appium	Android 디바이스 제어
Allure	HTML 리포트 생성 프레임워크
Playwright	HTML → PDF 변환 엔진
Slack API	메시지 + 파일 업로드

운영 주기

일일	Jenkins Cron 자동 수행
격주	정기 배포 전 회귀 테스트
월간	통계 자동 집계 (Slack → Excel)

업무 자동화 - 브레이즈 CRM 메시지 설정 검수 프로그램

프로젝트 배경

다른 팀 요청으로 개발 시작 (베타 버전)
CRM 메시지 설정의 정확성을 자동 검증하는 Python GUI 프로그램
Mac / Windows 크로스 플랫폼 지원 (app / exe 배포)

자동 검증 항목

일정 검증	1차/2차 일정 설정값과 실제 값 자동 비교
유저 수 체크	발송 대상 유저 12만 초과 시 경고 알림
랜딩 URL 검증	5개 링크의 리다이렉트 URL 정상 여부 확인
결과 출력	정상/비정상 케이스 실시간 UI 표시

동작 프로세스

CSV 로드 → Chrome 실행 → 일정 비교 → 유저 수 체크 → URL 검증 → 결과 출력
검증 결과는 Textbox에 실시간 출력, 정상/비정상 즉시 확인 가능

기술 구현

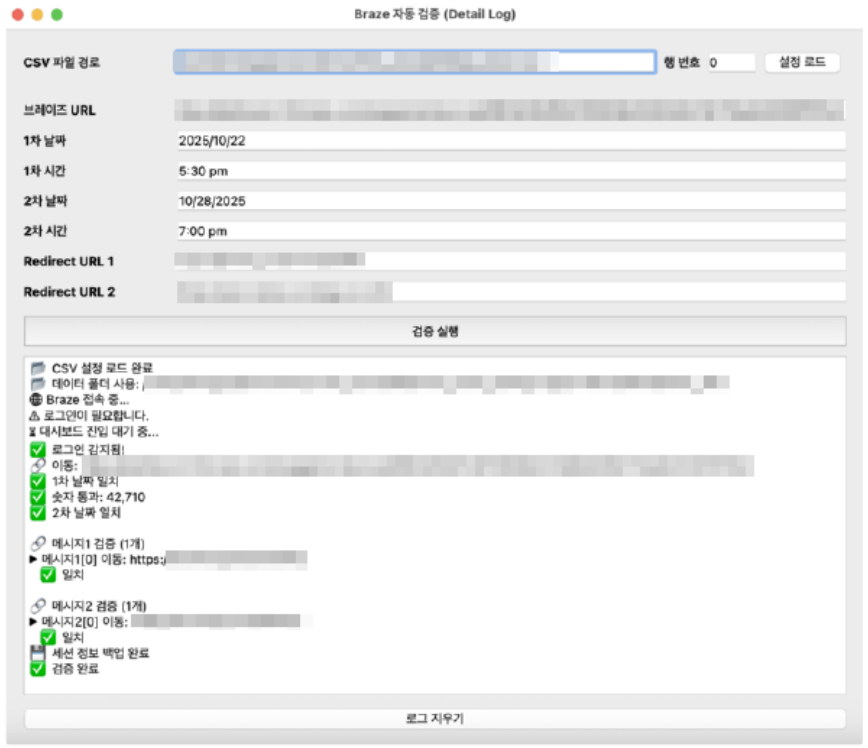
언어	Python + Tkinter GUI
웹 자동화	Selenium (Chrome) + 로그인 세션 유지
데이터 관리	CSV 기반 설정값 로드 / 자동 입력
배포	PyInstaller → exe (Win) / app (Mac)
검증 방식	설정값 비교 + URL 리다이렉트 체크

요구사항 상세

1. 1차/2차 일정 설정값 비교 (날짜+시간 조합 체크)
2. 발송 대상 유저 12만 초과 여부 체크
3. 메시지 내 랜딩 페이지 URL 정상 설정 확인
   a. 1차/2차 랜딩 페이지가 상이함
   b. 메시지에 링크 5개 포함
   c. 단축 링크 → 실제 페이지 이동 검증
4. 메시지 발송 테스트 (미구현 - 리스크 방지)

업무 자동화 - 브레이즈 CRM 메시지 설정 검수 - 실행 화면

정상 설정 케이스



설정이 모두 일치하는 경우

실행 화면 설명

GUI 인터페이스

Tkinter 기반 GUI로 비개발자도 쉽게 사용 가능
CSV 파일 선택 후 자동 검수 실행

실시간 검수 로그

검수 항목별 Pass/Fail 즉시 표시
오류 발견 시 상세 원인 + 위치 표시

자동 검증 항목

1차/2차 발송 스케줄 날짜 및 시간 검증
발송 종료 날짜 12시 초과 여부 체크
딥링크 URL 리다이렉트 정상 확인

결과 요약

전체 성공률 + 항목별 상세 결과 표시
성공/실패 케이스 실시간 UI 표시

크로스 플랫폼

PyInstaller로 exe(Win) / app(Mac) 배포
Mac/Windows 모두 동일 동작 보장

SUMMARY

핵심 강점 요약

1. **멀티 플랫폼 자동화 설계**
   Web(Playwright) + Android(Appium) + iOS(XCUITest) 동일 아키텍처로 3개 플랫폼 커버
2. **40개 시나리오 E2E 커버**
   Web 12개 + Android 14개 + iOS 14개, 비로그인→가입→로그인→구매→취소 전체 여정
3. **run_test_step 공통 패턴**
   관심사 분리 + 균일한 품질 + 템플릿 기반 확장, 전 플랫폼 동일 적용
4. **AppContext + CSV POM**
   Android 통합 컨텍스트 객체 + CSV 로케이터로 유지보수 최소화
5. **Fernet 암호화 계정 보호**
   .env + CSV 이중 암호화, 소스코드 내 평문 노출 제로
6. **Allure + Slack 리포팅**
   HTML 리포트 + Slack 실시간 알림으로 테스트 결과 즉시 확인
7. **CI/CD & 멀티 환경 지원**
   Web bat + Android Jenkins Device Farm + GUI 러너로 완전 자동화